An SMT Approach for Content-based Access Control

Nilanjan Ghosh
Indian Institute of Technology Delhi
nilanjan.ghosh@cse.iitd.ac.in

Kaustubh Beedkar
Indian Institute of Technology Delhi
kbeedkar@cse.iitd.ac.in

Abstract

Abstract goes here

ACM Reference Format:
Nilanjan Ghosh and Kaustubh Beedkar. 2018. An SMT Approach for Content-based Access Control. In *Proceedings of Make sure to enter the correct conference title from your rights confirmation email (Conference acronym 'XX).* ACM, New York, NY, USA, 4 pages. https://doi.org/XXXXXXX.XXXXXXX

1 Introduction

Introduction goes here

1.1 Policy Definition

In this paper we study the row level access for the relational databses. For a table *T*. an authorisation policy defines which tuples of T are accessible to a user at query time.

1.1.1 Policy Language. In this paper we have set a policy for a target table T as a boolean formula over atoms combined using $\vee/\wedge$.

Allowed atoms forms are

- col op const
- col1 op col2

where col, col1, col2 are columns and const is a constant and $op \in \{\neq, =, <, \leq, >, \geq\}$.

NULL semantics: if either operand is NULL, it is treated as false. It matches SQL *WHERE* filtering semantics.

1.2 Current technologies and problems with them

The predominant current mechanism in RDBMS is to rewrite the user query by inserting policy predicates into it, typically as additional *WHERE* clauses and extra joins/subquery for the cross table conditions. This works well for single table policies as simple single table predicates can be pushed down and using indexes gives additional benifits. But there are caveats when it comes when we move to cross table policies.

1.2.1 Core Limitations.

- **Policy - Query Entanglement**: Policies become part of the query, so query optimization and policy enforcement are no longer separable.
- **Cross table dependency**: Cross table policies introduce extra joins/subplans. When policies span multiple relations (chains, cycles, multiple join keys), the rewritten query can create large intermediate results or correlated subplans. So the optimizer may choose catastrophic plans, especially with many policies, OR branches, and mixed predicates.Every query repays the optimization and evaluation cost of policy logic embedded into it.

Another classic technique is to define a view $V_T = \sigma_{policy}(T)$(or more complex views) and force queries to reference views. It works well as it is conceptually simple and uses standard SQL semantics but it has it own limitations

- It is the same rewrite problem in disguise. Views are expanded/merged by the optimizer and effectively you still end up with a rewritten query.
- Many policies lead to many views. And cross table policies reintroduce joins/subqueries. In that case the view definition must include the witness relations, bringing back join amplification and plan fragility.

1.2.2 The limitation we exploit. The common failure mode is that rewriting/view approaches enforce policy by changing the query plan, so enforcement cost is dominated by query execution behavior (join order, intermediate sizes, correlated subplans), which becomes unpredictable and expensive as policies scale, especially for cross table policies.

We exploit the fact that for authorization, we do not need to compute join outputs. We only need existence of witnesses that make a tuple allowed. That is, for a tuple $t \in T$, a cross table condition is typically:

t is visible $\Leftrightarrow \exists$ witness tuples in other tables satisfying constraints with t.

This is an existential feasibility question and join materialization is an accidental byproduct of SQL execution, not a requirement for authorization semantics.

1.3 high level understanding and a diagram to make it easy

We model an instance of a relational database as a tripartite graph with three node layers:

- **Row IDs(RID layer)**: each tuple in each base table is a node. For a table T with n_T tuples, we denote its row nodes as,
$$R_T = \{r_0^T, r_1^T ....r_{n_{T-1}}^T\}$$
- **Values (Value layer)**: each distinct value appearing in policy relevant columns is represented as a node.
- **Attributes (Attribute layer)**: each policy relevant attribute is a node

Conference acronym 'XX, Woodstock, NY

ACM ISBN 978-1-4503-XXXX-X/2018/06
https://doi.org/XXXXXXX.XXXXXXX

Edges: We connect the layers with two edge types:

- **Row to value edges**: a row node connects to the value node of each attribute it carries. If tuple $r \in R_T$ has value x at attribute a, we add an edge
$$(r \rightarrow x)$$
- **Value to attribute edges**: each value node is connected to the attribute node(s) in which it occurs
$$(x \rightarrow a)$$

So every "cell" $r[a] = x$ induces a two hop path.
$$(r \rightarrow x \rightarrow a)$$

This graph view is convenient because it turns policy predicates into path constraints over the tripartite structure.

1.3.1 From value based nodes to position based nodes (tokens). : The tripartite representation described above is value based i.e. each distinct literal value (e.g., 10, 'F', 'AUTOMOBILE') is a node in the middle layer. This is conceptually clean, but it implicitly assumes that policy evaluation and join reasoning will be performed over raw values.
So our next optimization is a purely representational change. We keep the same tripartite graph semantics, but relabel every value-node by a small integer ID (its "position").

1.3.2 Tokenization as relabeling of the value layer. : Let the set of distinct non null values appearing in that domain be:
$$V = \{v_0, v_1 .... v_{m-1}\}$$
We define a mapping:
$$tok : V \rightarrow \{0, 1 .... m-1\}$$
where $tok(v_i) = i$. We call $tok(v)$ the token of value v.Intuitively, the token is the position of the value in the unique-value list.
So every edge of the form $(row \quad r) \rightarrow (value \quad v)$ becomes $(row \ r) \rightarrow tok(v)$
For ordered comparisons $\{<, \leq, >, \geq\}$ the value based graph requires comparing raw values. In the position based view we associate each token with its value's order via a rank:
$rank(tok(v))$ = the position of the v in a total order of the domain.
Then:
$v < v' \leftrightarrow rank(tok(v)) < rank(tok(v'))$

1.4 An example

Query: A simple join query between lineitem and orders

```
SELECT COUNT(*)
FROM lineitem l
JOIN orders  o ON l.l_orderkey = o.o_orderkey
WHERE l.l_shipmode = 'MAIL'
AND o.o_orderdate >= DATE '1996-01-01';
```

Two policies
Target: lineitem (cross table, includes equijoin + col–col inequality + col–const)

```
CREATE POLICY P_L
ON lineitem
FOR SELECT
USING (
    l_discount <= 0.07
    AND EXISTS (
        SELECT 1
        FROM orders o
        WHERE o.o_orderkey = l_orderkey
          AND o.o_orderstatus = 'F'
          AND l_extendedprice <= o.o_totalprice
    )
);
```

Target: orders (chain orders ,customer, nation, includes equijoins + col–const + col–col inequality)

```
CREATE POLICY P_O
ON orders
FOR SELECT
USING (
    EXISTS (
        SELECT 1
        FROM customer c
        JOIN nation n
          ON c.c_nationkey = n.n_nationkey
        WHERE c.c_custkey = o_custkey
          AND n.n_name IN ('GERMANY', 'FRANCE')
          AND o_totalprice != c.c_acctbal
    )
);
```

We'll treat both as permissive policies

1.5 Policies as path queries on the tripartite graph

1.5.1 A small "graph query language". Let

- $E_{\{T,a\}}(r, t)$ means row r of table T has token position t in the domain **a**.
- $Const(a, v)$ gives the token position of constant value v in the domain a.
- $Rank(a, t)$ gives an integer rank for ordered domains, used for $\{<, \leq, >, \geq\}$
- For any atom col1 op col2 we assume col1 and col2 belong to a shared comparable domain D and both values are tokenized by the same mapping tok_D.

Then we can write policies as path queries.

1.6 We convert the policies into a path query

Formally:

Lineitem policy: $P_L(l) \equiv \exists o, k, s, p_L, p_O, d \mid E_{L,orderkey}(l, k) \wedge E_{O,orderkey}(o, k) \wedge E_{O,orderstatus}(o, s) \wedge (s = Const(orderstatus, 'F')) \wedge E_{L,extendedprice}(l, p_L) \wedge E_{O,totalprice}(o, p_O) \wedge (Rank(price, p_L) \leq Rank(price, p_O)) \wedge E_{L,discount}(l, d) \wedge (Rank(discount, d) \leq Rank(discount, Const(discount, 0.07))$.

Orders policy:$P_O(o) \equiv \exists c, n, ck, nk, nm, pO, b \mid E_{O,custkey}(o, ck) \wedge E_{C,custkey}(c, ck) \wedge E_{C,nationkey}(c, nk) \wedge E_{N,nationkey}(n, nk) \wedge E_{N,name}(n, nm) \wedge (nm = Const(n\_name, 'GERMANY') \vee nm = Const(n\_name, 'FRANCE')) \wedge E_{C,c\_acctbal}(c, b) \wedge E_{O,totalprice}(o, pO) \wedge (pO \neq b)$.

1.7 Algebra for combining policies

Now in postgres we see permissive and restrictive policies. In our method we can essentially use any combination but for sake of comparison we restrict ourselves to postgres semantics of policy algebra.
For each target table T , let:

- permissive policies $P_T = \{p_1, p_2 \ldots . p_m\}$
- restrictive policies $R_T = \{r_1, r_2 \ldots . r_n\}$

Define:

$$\text{Perm}(T) = \bigvee_{p \in P_T} p; \ \text{Rest}(T) = \bigwedge_{r \in R_T} r; \ \text{Final}(T) = \text{Perm}(T) \wedge \text{Rest}(T).$$

1.7.1 Row-set semantics. : Let $[\phi]_T \subseteq \text{Rows}(T)$ be the set of target rows satisfying formula ϕ.
Then:

$$\text{Allow}(T) = [\text{Final}(T)]_T = \left(\bigcup_{p \in P_T} [p]_T \right) \cap \left(\bigcap_{r \in R_T} [r]_T \right)$$

In our running example,

- $Allowed(lineitem) = [P_L]_{lineitem}$
- $Allowed(orders) = [P_O]_{orders}$

And conceptually the query answer equals:

$$Q = \sigma_{\text{query\_preds}}\big((\text{Allow}(L) \bowtie \text{Allow}(O))\big)$$

but our system achieves this without rewriting by filtering scan outputs

1.8 The intent for Class Based approach

If you evaluate P_L or P_O literally as written, we are doing an existential join inside authorization:

- For each candidate lineitem row, look up matching orders, test status, test inequality extendedprice ≤ totalprice, etc.
- For orders, you chase a longer chain: orders →customer → nation, plus non equality o_totalprice ≠ c_acctbal.

So you don't want to "check predicates per row." You want to operate on classes/bins of rows that share the same token structure so you don't traverse the same logical edge type millions of times.

1.9 The class based approach

We now introduce a class based approach that avoids reasoning at individual tuple level. Intuitively tuples are first mapped to token as defined earlier, and then tuples that share the same relevant token pattern are grouped in to classes. Policy evaluation will operate over these classes rather than per row check.

1.10 How class based approach works

1.10.1 .

2 Experimental Evaluation

2.1 Experimental Setup

2.1.1 Datasets. We evaluate on one standard synthetic benchmark and three real-world datasets spanning four distinct domains (e-commerce, entertainment, transportation, and healthcare)

- **TPCH**: This is the synthetic dataset we are using which has been used widely for measuring query performance and is a common reference point in privacy/compliance systems work.
- **JOB / IMDb**: We use the Join Order Benchmark (JOB), constructed over a relationalized snapshot of IMDb.
  JOB paper (PVLDB 2015): https://www.vldb.org/pvldb/vol9/p204-leis.pdf
  JOB queries repo : https://github.com/gregrahn/join-order-benchmark
  Why we use it ?
  Real correlated data + join heavy workload. (good for multi table policies).
  It comes with workload (113 queries).
- **NYC TLC Trip Record Data**: We use the NYC Taxi trip records, a large real-world mobility dataset published by NYC TLC.
  NYC TLC Trip Record Data: https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page
  Taxi benchmark query directory :https://github.com/pdet/taxi-benchmark/tree/master/sql/queries
  Why we use it ? Very large data with time/location/fare attributes is a natural fit for scan-heavy analytics, making it ideal for evaluating "policy filter at scan" overheads
- **MIMIC**: Well it has many tables. No readymade queries so we can create really good benchmark queries

2.1.2 Queries. For TPCH and JOB we use the queries that comes with it. For MIMIC we use 25 carefully crafted queries designed by us. For NYC Taxi It comes with 4 queries but we can expand on it.

2.1.3 Baselines. We compare against representative enforcement mechanisms for row-level authorization that are standard in the FGAC literature.

- **No policy**
  We run each query without any access control enabled to obtain the best-case execution time for the underlying workload.
- **Native PostgreSQL RLS**
  – with indexing
  – without indexing
- **Authorization Views**
- **Sieve**

2.2 Policies

: Each predicate in our policies in each dataset are in the form

- **col op col**
- **col op const**

where op ∈ {=, ≠, <, ≤, >, ≥}.
For each dataset, we instantiate 20 policies.

- 10 single table policies
- 10 multi table policies

To systematically vary difficulty, we control policy complexity by the number of predicates. To ensure policies are meaningful for the workload, we anchor them to (i) the largest tables and (ii) common join paths used by the benchmark queries

- **TPCH**: single table policies are defined on LINEITEM and CUSTOMER . Multi table policies span common join paths such as LINEITEM X ORDERS X CUSTOMER and longer chains through SUPPLIER/NATION/REGION.
- **JOB/IMDb**: single-table policies target large tables such as cast_info, movie_info, and title, while multi-table policies align with JOB join patterns (e.g., title X cast_info X name, title X movie_info X info_type).
- **NYC Taxi**: single-table policies target the trip table (e.g., Yellow/Green trips). Multi table policies join trips with a location/zone dimension (e.g., pickup/dropoff location to zone/borough).
- **MIMIC**: single-table policies target large tables (e.g., labevents, chartevents) . Multi-table policies follow common clinical join paths (e.g., admissions X diagnoses_icd, icustays X labevents, admissions X prescriptions)

We implement the same policy bunch for all baselines

2.2.1 Database Engine. We will use postgres 18 for evalution on a system with specs as followed

`will be added later`

2.2.2 Design. For each dataset we will create a *run.csv*. Across datasets the common headers in the csv is

- baseline
- query_id
- num_policies. the total number of policies that are active when the query is fired
- query_complexity. `We need to define it`
- policy_complexity . it is the number of predicates that is applied to the query from the policy side
- cold_ms . The time taken to do the first fetch
- hot_ms . it is the avg of 5 consecutive runs for the query after cold start.
- correctness . It is a binary value that is true if the returned rows by the query is same as the rewritten version of the query.
- mem_overhead . Extra space needed to execute the technique. It is relevant with our technique and RLS with indexing.
- status . 1 if the query executed and 0 if it timed out (30 minutes) or faced error from database
- pre_run_memory_building_ms . It is the time taken for pre run preparation. It is relevant with our technique and RLS with indexing. `Although I am not confident about this attribute in the csv`

In the tpch run we will introduce another header, scale_factor that will vary between 0.1, 1, 10, 100.

2.2.3 Extra. We will use the same tpch evaluation and add a header, variant which will help demonstrate the efficiency between value based, position based and class based approach. our solution variant is by defualt the class based one as it is very fast.

2.3 Experimental Result

References